SlabTrade

Technical Stack & Architecture Reference

Monorepo: `slab-trade` | Generated 2026-09-09 | Source: repository `main`

1. Overview

SlabTrade is a B2B marketplace for trading natural stone slabs (granite, marble, quartzite). It connects **vendors** (sellers) who publish slab lots with **customers** (buyers), under oversight of **admin** and **support** roles. The codebase is a TypeScript monorepo split into a REST API (`apps/api`) and a Next.js web client (`apps/web`), with shared config/UI packages.

TypeScript | pnpm workspaces | Turborepo | Express 5 | Prisma + PostgreSQL | Next.js 16 | React 19 | Tailwind CSS v4 | shadcn/ui | Zod | JWT

2. Monorepo & Tooling

Concern	Technology	Notes
Package manager	`pnpm@9` workspaces	Workspace globs: `apps/*`, `packages/*`. Node `>=18`.
Task runner	Turborepo 2.x	Pipelines for `dev`, `build`, `lint`, `check-types` with dependency-aware caching; TUI output.
Language	TypeScript 5.9.2	Strict typing across every app and package; path aliases per package.
Linting	ESLint 9 (flat config)	Shared via `@repo/eslint-config`; `typescript-eslint`, `eslint-plugin-import`, `eslint-config-prettier`.
Formatting	Prettier 3.7	Repo-wide `format` script over `ts,tsx,md`.
Shared packages	`@repo/ui`, `@repo/eslint-config`, `@repo/typescript-config`	Reusable React components and base `tsconfig` presets (base / nextjs / react-library).

3. Backend — `apps/api`

A modular REST API built on Express 5 and TypeScript, following a controller → service → repository layering with a bootstrap layer that assembles the app.

3.1 Runtime & Framework

Package	Version	Purpose
`express`	^5.2	HTTP server and routing.
`tsx`	^4.23	Dev runtime with watch mode (`tsx watch`) — no separate build step in development.
`typescript` + `tsc-alias`	5.9 / ^1.9	Production build: `tsc` compiles to `dist/`, `tsc-alias` rewrites path aliases.
`tsconfig-paths`	^4.2	Resolves `@modules/*`, `@common/*`, `@bootstrap/*`, `@config/*` aliases at runtime.
`dotenv`	^17	Environment loading; typed access via `src/config/env.ts`. Default port `5000`, API prefix `/api`.

3.2 Data Layer

Package	Version	Purpose
`prisma` / `@prisma/client`	6.16.3	ORM and type-safe query client (`prisma-client-js` generator).
PostgreSQL	—	Primary datastore; connection via `DATABASE_URL`.
Prisma Migrate	—	Versioned schema migrations under `prisma/migrations/` (auth init, refresh-token family, token revocation, ...).
Seeding	`tsx prisma/seed.ts`	Seeds roles and baseline data.

Models: `User` (soft-deletable, verification flags, profile image), `Role` (RBAC), `RefreshToken` (rotation with `familyId` + revocation columns), `SlabListing` (material, thickness, length/width ranges, `Decimal(10,2)` unit price, colour category, vein pattern, image array, active/inactive status, lots-per-container).

3.3 Authentication & Security

Package	Version	Purpose
`jsonwebtoken`	^9.0	Access-token issuing/verification (`JWT_SECRET`, `JWT_EXPIRES_IN` default 7d).
`bcrypt`	^6.0	Password hashing.
`cookie-parser`	^1.4	Reads refresh-token cookies.
Refresh-token rotation	—	Token families with replacement tracking and revocation to detect reuse.

`helmet`	^8.3	Security response headers.
`cors`	^2.8	Cross-origin access for the web client.
`express-rate-limit`	^8.6	Request throttling (`rateLimitMiddleware`).
Auth/role middleware	—	`authMiddleware`, `roleMiddleware` guard protected routes by role.

3.4 Validation, Uploads, Observability

Package	Version	Purpose
`zod`	^4.4	Request-schema validation per module (`*.schema.ts`) via a `validate` middleware.
`multer`	^2.2	Multipart slab-image uploads; served statically from `/uploads` with cross-origin resource policy.
`pino` / `pino-http` / `pino-pretty`	^10 / ^11 / ^13	Structured JSON logging with pretty dev output.
`morgan`	^1.11	Concise HTTP request logging in dev.
`compression`	^1.8	gzip response compression.
`swagger-jsdoc` + `swagger-ui-express` + `yamljs`	^6 / ^5 / ^0.3	OpenAPI docs generated from JSDoc annotations (`*.swagger.ts`), served as interactive UI.
`uuid`	^14	Identifier generation.

3.5 Structure

- `src/modules/` — feature modules: `auth`, `users`, `slabs`, `admin`, `health`. Each groups controller, service, repository, routes, Zod schema, Swagger, and types.
- `src/common/` — cross-cutting: `auth` (bcrypt, JWT middleware), `errors`, `logger`, `middlewares` (validate, rate-limit, role, error handler, not-found), `responses` (standard envelope), `utils`.
- `src/bootstrap/` — app assembly: `express`, `middleware`, `routes`, `swagger`, `database`.
- `src/config/` — `env.ts`, `app.config.ts`.

4. Frontend — `apps/web`

A Next.js App Router application (React Server Components) serving the public marketing site plus role-scoped dashboards for vendors, customers, admins, and support.

4.1 Framework & UI

Package	Version	Purpose
`next`	16.3.0	App Router, RSC, server actions; dev on port `3000`. (Note: this Next major carries breaking changes vs. earlier versions.)
`react` / `react-dom`	19.2	UI runtime with the React 19 feature set.
`tailwindcss` + `@tailwindcss/postcss`	^4.3	Utility-first styling, Tailwind v4 (CSS-config, no `tailwind.config`); `tw-animate-css` for animation utilities.
`shadcn` / `radix-ui`	^4.18 / ^1.6	Component system (`radix-nova` style, neutral base, CSS variables) built on Radix primitives — `components/ui/*`.
`class-variance-authority`, `clsx`, `tailwind-merge`	—	Variant-driven component styling and class merging.
`lucide-react`	^1.33	Icon set.
`next-themes`	^0.4	Light/dark theme switching.
`motion`	^13.1	Animations / transitions (`components/motion`).
`embla-carousel-react`	^8.6	Carousels (slab imagery, marketing).
`@tanstack/react-table`	^8.21	Headless data tables for dashboard listings.
`sonner`	^2.0	Toast notifications.
`zod`	^4.4	Client-side form validation (`lib/validations`), mirroring API schemas.

4.2 App Structure & Data Access

- `app/(marketing)/` — public routes: home, `buyer`, `seller`, `contact`.
- `app/login`, `app/register` — auth screens; `app/admin`, `app/vendor` (`upload-slabs`, `my-slabs`), `app/customer`, `app/support` — role dashboards with their own layouts.
- `lib/api.ts` — typed `fetch` wrapper: base URL from `NEXT_PUBLIC_API_URL`, bearer-token header, FormData-aware content type, normalised error handling.
- `services/` — `auth.service.ts`, `slab.service.ts`, `user.service.ts` call the API.
- `lib/auth/` — `auth-guard`, `auth-storage`, `role-redirect` for client-side session and role routing.
- `components/` — `ui` (shadcn), `site`, `dashboard`, `auth`, `motion`; `hooks/use-mobile.ts`.

5. Cross-Cutting Choices

- **End-to-end TypeScript** with Zod as the single validation vocabulary on both tiers.
- **Stateless auth** — short-lived JWT access tokens plus rotating refresh-token families persisted in PostgreSQL for reuse detection.
- **Separation of concerns** — the API owns all persistence and business rules; the web app is a pure consumer via `lib/api.ts`.
- **Consistent API envelope** — `common/responses` and a central error handler produce uniform success/error shapes; OpenAPI generated from code.
- **Turborepo caching** keeps `build` / `lint` / `check-types` incremental across the workspace.

> This document describes the technology stack only. For product scope, data model detail, API contracts, and deployment, see the companion documents under `docs/`.